实验一：系统分析及单例模式

实验与创新实践教育中心 • 计算机与数据技术实验教学部

各实验班安排

课程名称	上课时间	实验室
软件构造（吴老师第1组）	3-6周|星期3|5-6节	T2608
	7周|星期2|1-2节	T2507
	7周|星期4|11-12节	T2608
	9周|星期5|5-8节	T2608
软件构造（吴老师第2组）	3-6周|星期3|1-2节	T2507
	7周|星期3|5-8节	T2210
	8周|星期3|5-8节	T2507
软件构造（张老师第2组）	3周|星期3|7-8节	T2608
	4-5,6周|星期5|5-6节	T2608
	7周|星期6|5-8节	T2608
	9周|星期3|5-8节	T2210
软件构造（张老师第1组）	3周|星期3|7-8节	T2507
	4周|星期3|7-8节	T2210
	5周|星期4|1-2节	T2210
	6周|星期4|1-2节	T2507
	7-8周|星期3|5-8节	T2102

*如有课程冲突需要调整的请与我联系

实验注意事项

1. 准时上课，调课或请假请至少提前3天跟我沟通；
2. 课堂会有不定时签到、提问，若缺课1/3取消实验成绩；
3. 每次实验环环相扣，请跟上进度，迟交会扣延迟分；
4. 下课请关机，带走垃圾，最后离开的同学请关空调、关灯、关门。

本学期实验总体安排

初始版本

最终版本

初始版本

游戏主界面
英雄机移动
英雄机子弹直射
碰撞检测
统计得分和生命值

01

UML建模
精英敌机生成、攻击
道具生成、加血道具生效
采用**单例模式**创建英雄机

02

采用**简单工厂模式**
创建道具
采用**工厂模式**
创建敌机

03

采用**策略模式**
实现不同飞机的发射弹道
两种火力道具生效

04

采用**数据访问对象模式**
实现得分排行榜
添加**JUnit单元测试**

05

使用**Swing**添加游戏难度选择和排行榜界面
使用**多线程**实现音效控制及火力道具状态恢复

06

采用**观察者模式**
实现炸弹、冰冻道具
采用**模板模式**
实现三种游戏难度

本学期实验总体安排

实验项目	一	二	三	四	五	六
学时数	2	2	2	2	4	4
实验内容	**系统分析** **单例模式**	简单工厂 工厂模式	策略模式	JUnit 数据访问 对象模式	Swing 多线程	观察者模式 模板模式
时间节点			**中期**			**结项**
提交内容			UML类图 项目代码			结项报告 代码、类图 展示视频

实验课程共 16 个学时，6 个实验项目，总成绩为 30分 。

目录

CONTENTS

01 实验目的

02 实验任务

03 实验步骤

04 迭代目标

实验目的

01

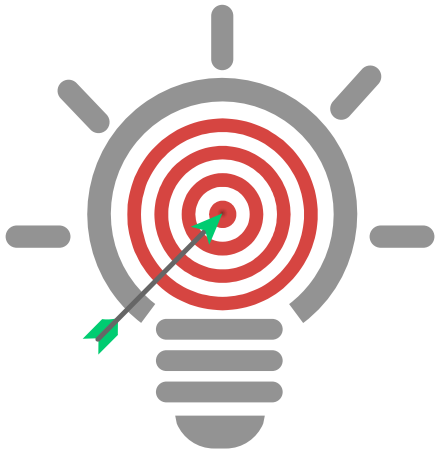
面向对象思想

理解类、对象、继承等基础概念。

02

分析与设计

从飞机大战需求中抽取核心实体与关系。



03

UML 类图建模

用 PlantUML 表达系统继承结构。

04

单例模式应用

完成英雄机单例设计与代码重构。

实验任务

01 跑通模板程序

理解系统功能，导入并运行模板程序。

02 完成继承类图，扩展核心实体

分析核心实体类关系，用 PlantUML 绘制完整类图；新增 4 种敌机、5 种道具及其抽象父类。

03 绘制单例结构图，重构创建逻辑

理解单例模式，并完成英雄机单例 UML 建模；重构英雄机创建代码。

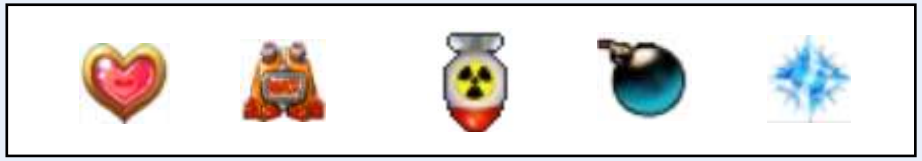
实验步骤：飞机大战任务书（1）

角色设定

- 英雄机

- 敌机

- 道具



- 子弹

实验步骤：飞机大战任务书（2）

游戏规则

- 游戏难度：简单、普通、困难
- 子弹发射、弹道变化
- 道具生成、使用和效果
- 生命值和得分计算、总分排行榜

实验步骤：飞机大战任务书（3）

界面和音效

- 游戏地图
- 音效的开启和关闭

需求细则请参考实验指导书！

实验步骤：模板程序导入及运行（1）

➢ 本实验提供了飞机大战的模板代码 AircraftWar-base，已实现如下功能：

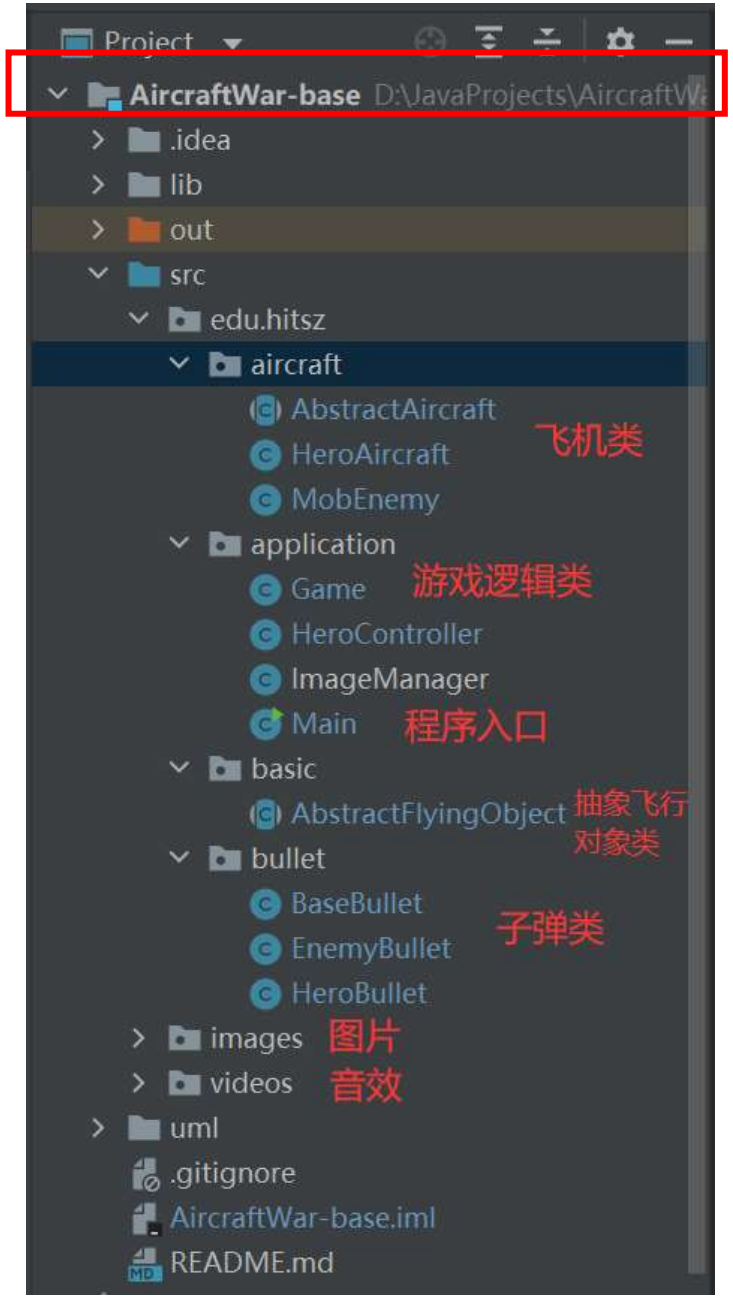
- 游戏主界面
- 英雄机、普通敌机移动
- 英雄机子弹直射
- 碰撞检测
- 统计并显示得分和英雄机生命值

实验步骤：模板程序导入及运行（2）

- 下载并安装java集成开发环境 IntelliJ IDEA

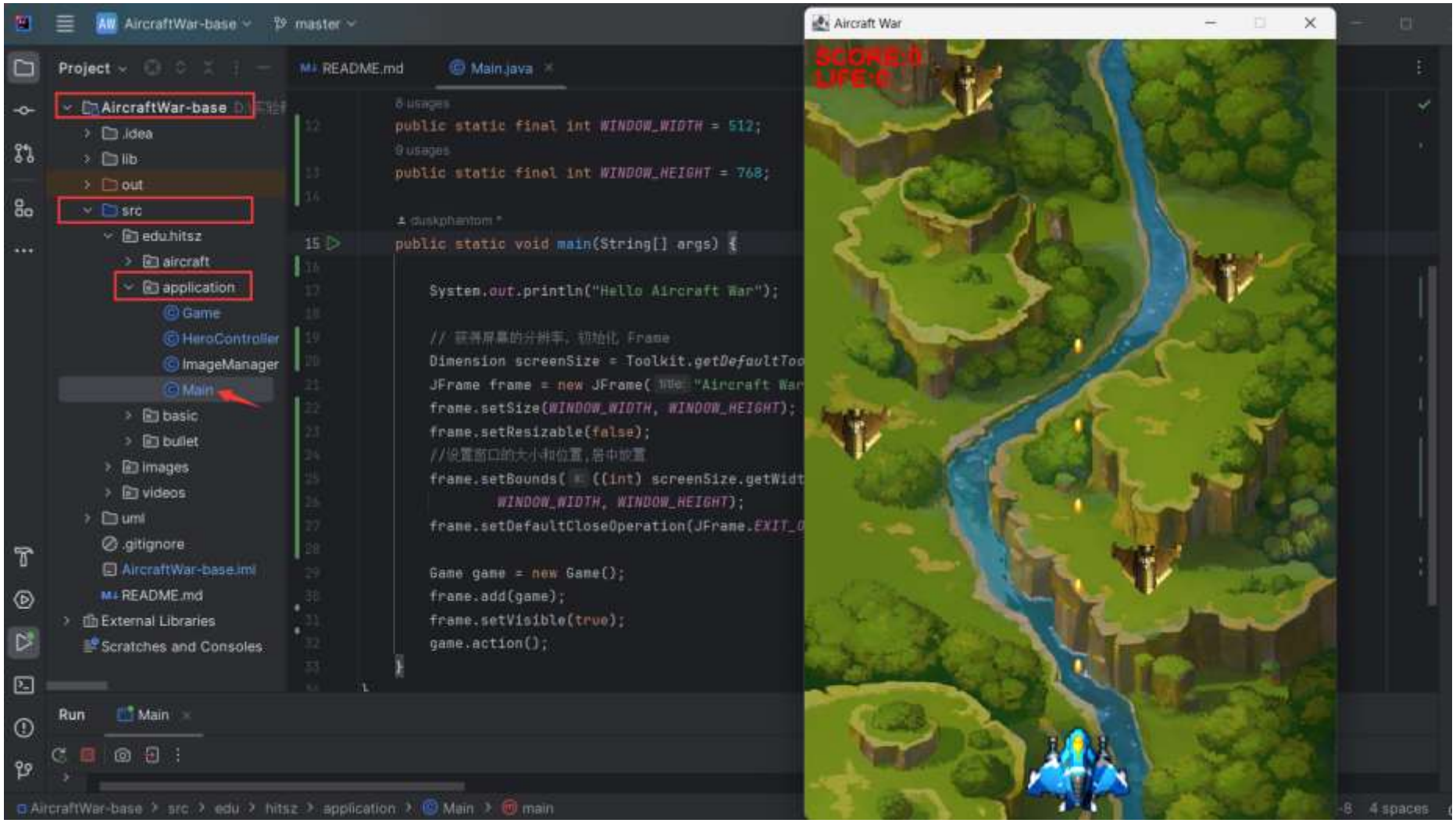
https://www.jetbrains.com/zh-cn/idea/download/#section=windows

- 导入项目AircraftWar-base



实验步骤：模板程序导入及运行（3）

➢ 运行Main.java



AircraftWar 框架代码快速上手

先看懂这 3 件事

1. **框架已经帮你做了什么**
2. 程序从哪里启动、怎么一轮轮跑起来
3. 新功能通常会改哪些位置

建议阅读顺序：

Main → Game 构造函数 → action()

→ 具体逻辑方法

先盯住入口和主循环，后面加功能就不会乱。

框架已经帮你做了什么

1 **基础界面** 主窗口、滚动背景、定时刷新与 repaint() 已打通。

2 **核心玩法** 英雄机跟随鼠标移动；英雄机会周期射击；普通敌机会周期生成。

3 **结果反馈** 子弹可命中敌机并加分；界面已显示 SCORE / LIFE；英雄机死亡后结束。

当前已经能看到

能运行 能移动 能碰撞

- 英雄机随鼠标走
- 普通敌机向下飞
- 子弹出界自动失效

程序从哪里启动

1 **Main.java**

创建 JFrame，把 Game 面板加入窗口，最后调用 game.action()。

```
Game game = new Game();   game.action();
```

2 **Game.java**

主游戏面板：维护 hero / enemy / bullet 列表，也负责 paint() 与主要逻辑方法。

3 **action()**

定时执行：生成敌机 → 射击 → 移动 → 碰撞 → 后处理 → repaint()。

建议阅读顺序

1 先看 Main.main()
2 再看 Game 构造函数
3 然后进入 action()
4 最后看 shootAction() / crashCheckAction() / postProcessAction()
5 遇到输入和图片，再回头看 HeroController / ImageManager

包结构和文件职责

1. **application 包** — Main、Game、HeroController、ImageManager：入口、主循环、输入控制、图片管理。
2. **aircraft + bullet** — HeroAircraft、MobEnemy、BaseBullet 等核心对象都在这里扩展。
3. **basic 包** — AbstractFlyingObject：位置、速度、碰撞、失效标记等公共能力。
4. **资源文件** — images / videos / uml：图片、音频、类图文件都在资源目录里。

项目目录

```
src/
├─ edu/hitsz/application
├─ edu/hitsz/aircraft
├─ edu/hitsz/basic
├─ edu/hitsz/bullet
├─ images
├─ videos
└─ uml
```

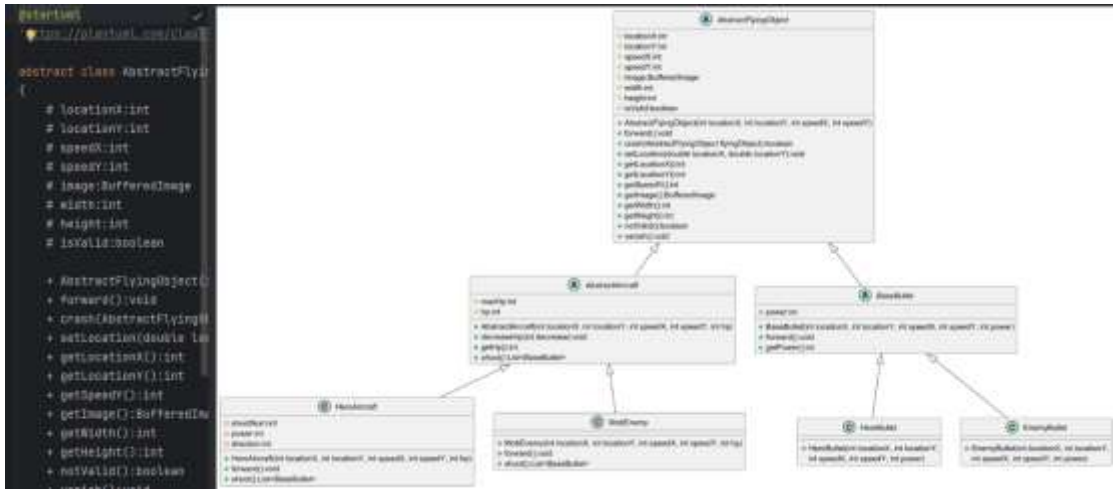
实验步骤：核心实体类设计与UML建模（1）

设计范围

4 类核心实体 • 共 13 类

- 英雄机 × 1种
- 敌机 × 5种
- 道具 × 5种
- 子弹 × 2种

PlantUML



01 识别共性

提炼位置、速度、血量、碰撞等公共属性

02 构建层次

抽象父类 + 具体子类，形成清晰继承结构

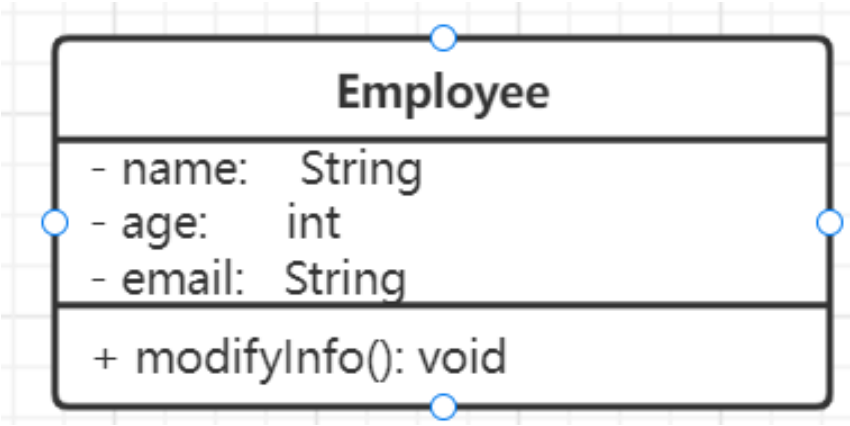
03 完成类图

在 uml/Inheritence.puml 中补充完整 UML

交付要求：完整展示具体类、抽象父类及继承关系，层次结构清晰、便于后续扩展。

实验步骤：核心实体类设计与UML建模（2）

类图（Class Diagram）：用来显示系统中的类、接口以及它们之间的静态结构和关系的一种静态模型。




```
public class Employee {

    private String name;
    private int age;
    private String email;

    public void modifyInfo() {
        ......
    }
}
```

实验步骤：核心实体类设计与UML建模（3）

1 泛化关系

一句话理解 **是一个（继承）**

子类继承父类的属性和行为，通常对应 extends。

例子 老虎是动物；EliteEnemy 是 Enemy

2 实现关系

一句话理解 **按接口约定去做**

类把接口中约定的方法真正实现出来，通常对应 implements。

例子 支付类实现 Pay 接口

3 依赖关系

一句话理解 **临时要用一下**

一般出现在一个类的方法参数、局部变量或返回值。

例子 驾驶员开汽车，方法里临时创建 Car

4 一般关联

一句话理解 **长期认识 / 长期持有**

一个对象长期知道另一个对象，常见为成员变量引用。

例子 老师管理学生；班级知道学生列表

5 聚合关系

一句话理解 **整体 - 部分（可拆分）**

部分离开整体后仍能独立存在，重点看“能不能单独活”。

例子 汽车和轮胎

6 组合关系

一句话理解 **整体 - 部分（不可拆分）**

部分离开整体就失去独立意义，比聚合更紧密。

例子 学校与教务处

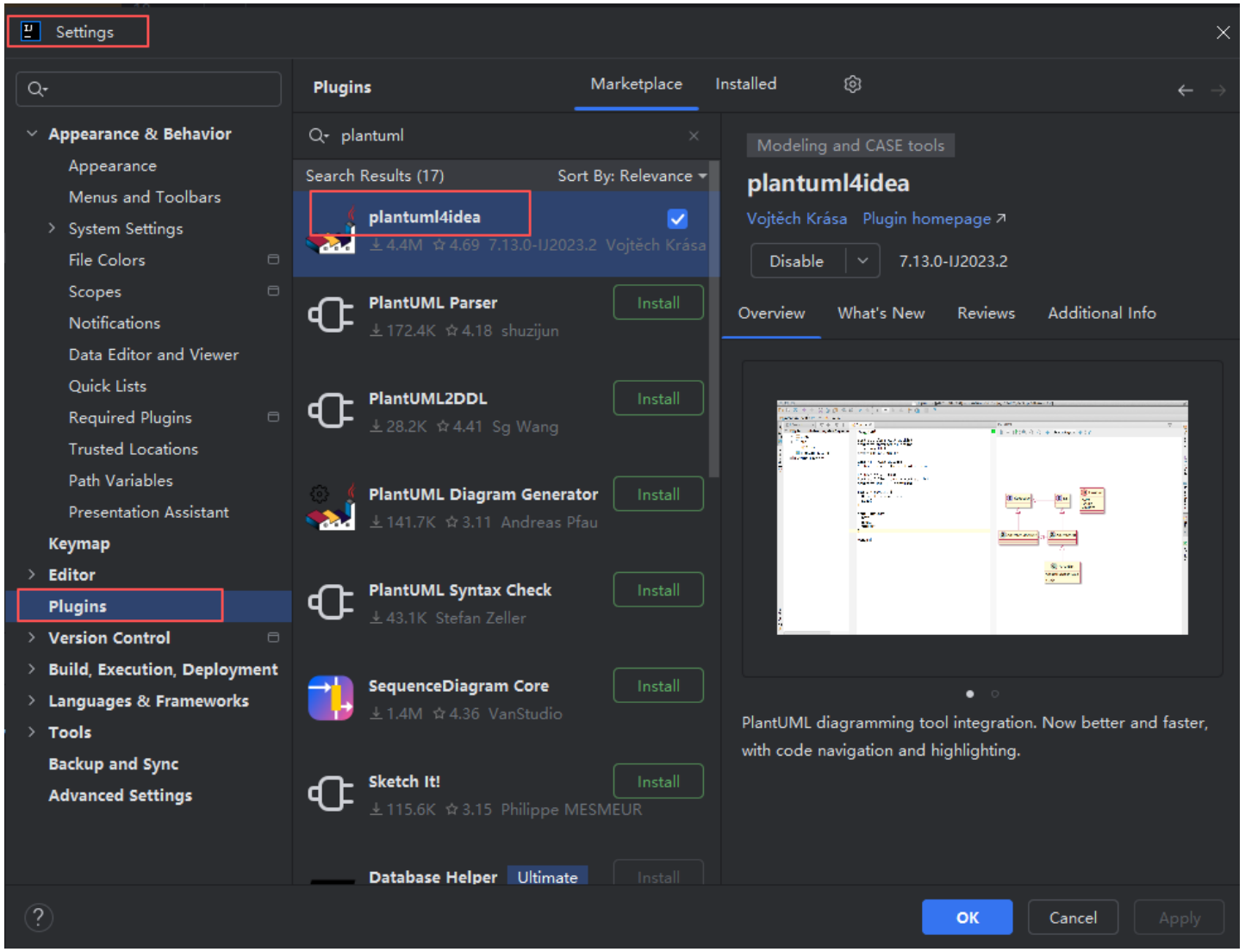
“是一个” → 泛化 / 实现

“有一个” → 关联 / 聚合 / 组合

“临时要用” → 依赖

实验步骤：PlantUML插件（1）

- **PlantUML**是一个开源项目，支持快速绘制类图、时序图、用例图等UML结构图和行为图。



实验步骤：PlantUML插件（2）

- **PlantUML** 类图及相关语法请参考官网说明：
  https://plantuml.com/class-diagram

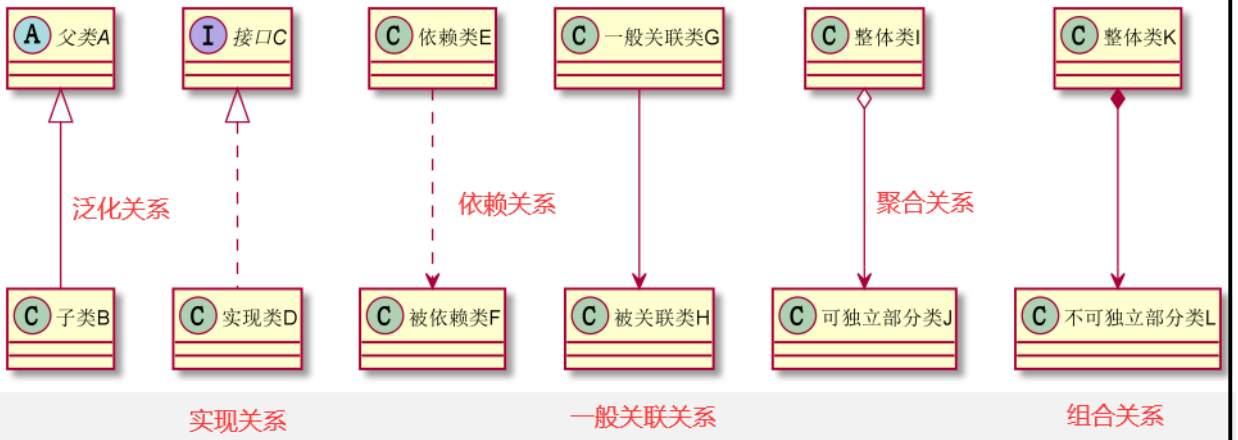
- 接口、抽象类、类的画法：

```
@startuml
'https://plantuml.com/class-diagram

interface 接口
{
}
abstract class 抽象类
{
}

class 类
{
    + 公有成员变量: String
    - 私有成员变量: int
    # 受保护的成员变量: Date
    + {static} 静态成员变量: String

    + 构造函数(int 参数1)
    + 普通方法(int 参数1, String 参数2): void
    + {abstract} 抽象方法(): int
    + {static} 静态方法(int 参数1, String 参数2): String
}
```

实验步骤：PlantUML插件（3）

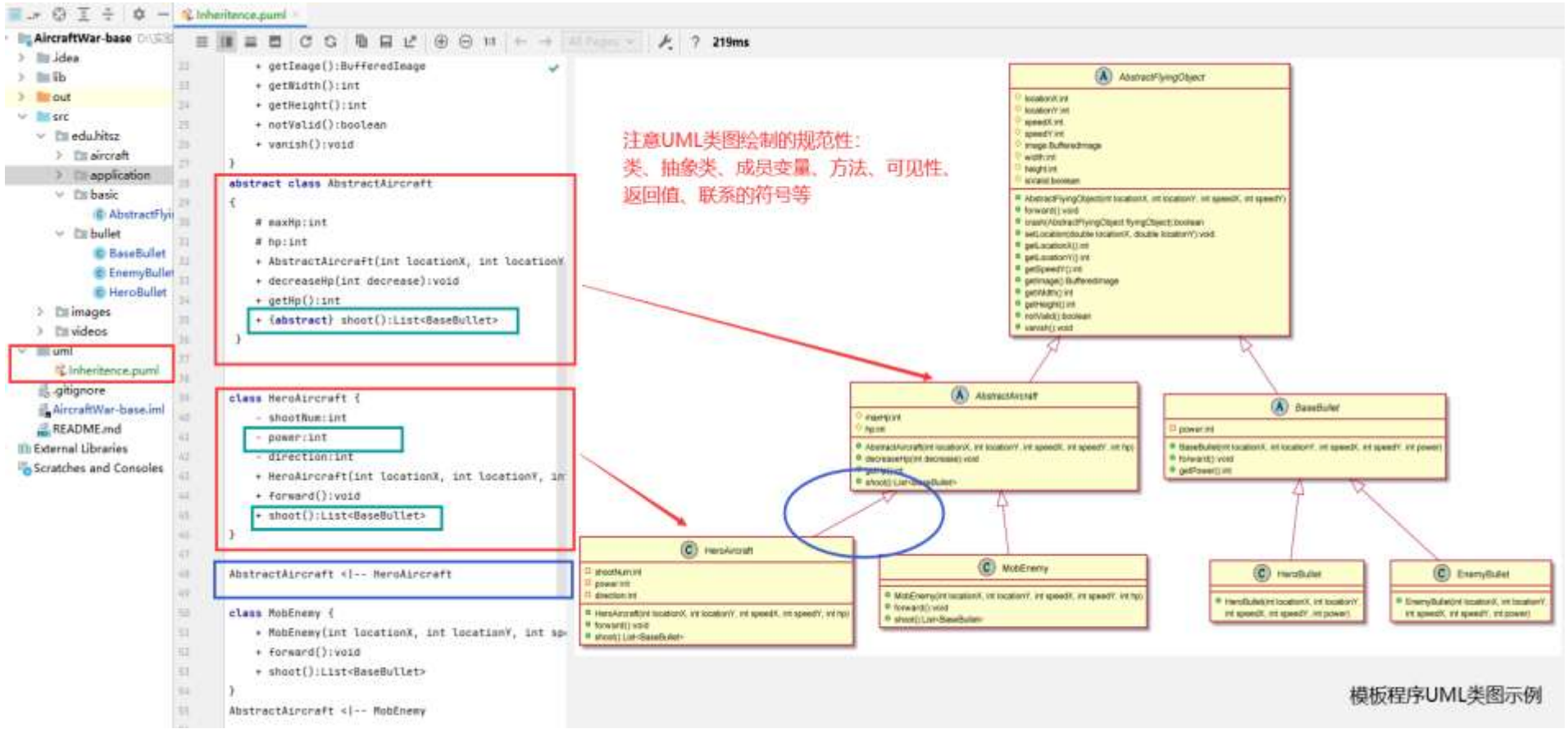
➢ 类与类之间6种关系的画法：

```
@startuml
'https://plantuml.com/class-diagram

abstract class 父类A
class 子类B
父类A <|-- 子类B

interface 接口C
class 实现类D
接口C <|.. 实现类D

class 依赖类E
class 被依赖类F
依赖类E ..> 被依赖类F

class 一般关联类G
class 被关联类H
一般关联类G --> 被关联类H

class 整体类I
class 可独立部分类J
整体类I o--> 可独立部分类J

class 整体类K
class 不可独立部分类L
整体类K *--> 不可独立部分类L
```



实验步骤：PlantUML插件（4）

➢ 注意：请严格按照指导书 4.3 节要求绘制 UML 类图。



可通过在箭头内部使用关键字**left, right, up** 或 **down**来改变方向。（如：foo -left-> dummyLeft）

实验步骤：英雄机场景分析（1）

在飞机大战游戏中只有一种英雄机，且每局游戏只有一架英雄机，由玩家通过鼠标控制其移动。

```
public Game() {
    heroAircraft = new HeroAircraft(
            locationX: Main.WINDOW_WIDTH / 2,
            locationY: Main.WINDOW_HEIGHT - ImageManager.HERO_IMAGE.getHeight() ,
            speedX: 0,  speedY: 0,  hp: 1000);
```

目前能否保证英雄机的唯一性？

不能，外部程序可以随意用new方法创建一个实例。

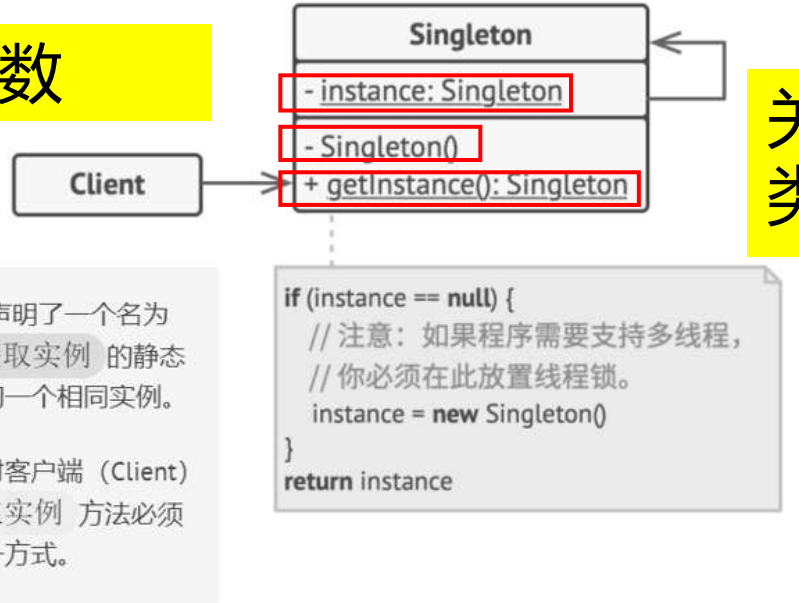
实验步骤：单例模式结构图

单例模式（Singleton Pattern）是一种创建型设计模式， 能够保证一个类只有一个实例， 并提供一个访问该实例的全局节点。

关键点1：私有静态变量

关键点2：私有构造函数

关键点3：提供一个访问该类唯一实例的方法



单例模式结构图

实验步骤：单例模式示例代码

① 饿汉式

```java
public class EagerSingleton {
    private static EagerSingleton instance = new EagerSingleton ();
    private EagerSingleton (){}
    public static EagerSingleton getInstance() {
        return instance;
    }
}
```

② 懒汉式

```java
public class LazySingleton {
    private static LazySingleton instance = null;
    private LazySingleton (){}
    public static synchronized LazySingleton getInstance() {
    if (instance == null) {
        instance = new LazySingleton();
    }
    return instance;
    }
}
```

③ 双重检查锁定（DCL）

```java
public class Singleton {
    private volatile static Singleton singleton;
    private Singleton (){}
    public static Singleton getSingleton() {
        if (singleton == null) {
            synchronized (Singleton.class) {
                if (singleton == null) {
                    singleton = new Singleton();
                }
            }
        }
        return singleton;
    }
}
```

本次迭代开发目标（1）

注意：本次迭代仅需完成下列清单任务，实现框架或模拟输出，任务书中其余细节功能将在后续迭代中逐步完善。

任务清单
(CheckList)

UML类图设计

① 完善系统继承关系类图、绘制单例模式结构图

设计模式应用

② 重构代码，采用单例模式创建英雄机

核心实体类扩展

③ 在项目中增加其余4种敌机、5种道具以及它们的抽象父类，完善继承结构

（仅需完成类定义，无需实现复杂逻辑和功能）

敌机生成

④ 界面上可观察到普通和精英敌机按周期随机出现

本次迭代开发目标（2）

注意：本次迭代仅需完成下列清单任务，实现框架或模拟输出，任务书中其余细节功能将在后续迭代中逐步完善。

敌机攻击

⑤ 精英敌机按设定周期向下直射单排子弹

道具生成

⑥ 精英敌机坠毁时，以一定概率随机生成一个道具（范围限定为：加血道具、火力道具、超级火力道具）

道具生效

⑦ 英雄机拾取道具后，道具自动生效并消失：加血道具可使英雄机恢复一定血量，但不超过初始最大值；两种火力道具无需具体实现，生效时只需在控制台打印 **FireSupply active!** 或 **FirePlusSupply active!** 即可。

作业提交

- 本次无需提交

- 中期检查的时候将项目代码和UML类图打成zip包，提交至作业提交平台

  作业平台入口：
  https://grader.hitsz.edu.cn/login?next=/courses

  - 用统一身份认证登录
  - 推荐使用 Chrome 浏览器

同学们，请开始实验吧！

THANK YOU